POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY
Faculty of Information Technology

Intelligent System Development

ASSIGNMENT 07

A Five-Layer RNN and a Five-Layer LSTM as Compositions of Functions: PyTorch and Keras on Review Sentiment and Stock Volatility

Instructor / Lecturer: Assoc. Prof. Dinh Que Tran, Ph.D.
Email: quetd@ptit.edu.vn, tdque@yahoo.com

Student Name: Nguyễn Văn Trường
Student ID: B23DCCE095
Semester: I.2025 – 2026
Class: E23CNPM02
Submission: Individual Project

Hanoi, 2026

Contents

1 Overview **2**

2 A Five-Layer RNN and a Five-Layer LSTM as Compositions of Functions **3**
- 2.1 The Five Elementary Functions . . . 3
- 2.2 One RNN Cell . . . 3
- 2.3 Stacking Five Layers . . . 4
- 2.4 Counting Parameters . . . 5
- 2.5 The Gradient Through Depth and Time . . . 5
- 2.6 The LSTM Cell . . . 6
- 2.7 Gradient Flow in the LSTM . . . 7

3 Data **8**
- 3.1 EC: Amazon Reviews of Unlocked Mobile Phones . . . 8
- 3.2 ST: S&P 500 Stocks . . . 10

4 Models, Protocol and Implementation **11**
- 4.1 Architecture and Hyperparameters . . . 11
- 4.2 Keras and PyTorch . . . 11
- 4.3 Baselines and Metrics . . . 12

5 Experiment 1: Three-Way Sentiment of Reviews (EC) **12**

6 Experiment 2: One-Week Stock Volatility (ST) **15**

7 Comparison: LSTM against RNN, Keras against PyTorch **17**

8 Conclusions and Limitations **17**

A Reproducing the Results **18**

1 Overview

The assignment asks for four things: the structure of a 5-layer RNN and of a 5-layer LSTM written as compositions of elementary functions and explained; the two networks implemented in PyTorch and in Keras; two datasets (comments from an e-commerce site and stock prices); and a run, evaluation and comparison of LSTM against RNN on both. This report does all four with the same protocol for every run. The datasets are **EC**, Amazon reviews of unlocked mobile phones (162,490 unique reviews, classified as negative, neutral or positive), and **ST**, S&P 500 stocks (145 stocks, 2010–2024, forecasting the mean absolute move of the next five trading days). Each dataset is run with 2 architectures × 2 frameworks = 4 runs, named `<dataset>-<framework>-<model>`, for example `EC-PT-LSTM` (`TF` = Keras/TensorFlow, `PT` = PyTorch).

Table 1: All eight runs on the test sets. EC: 16,249 reviews; ST: 108,315 windows (2022–2024). s/epoch = median time per epoch on a CPU.

Run	main metric	value	second metric	params	s/epoch	train time (s)
`EC-TF-RNN`	macro-F1	0.614	acc. 0.829	760,451	69.7	286
`EC-PT-RNN`	macro-F1	0.605	acc. 0.830	760,771	143.4	566
`EC-TF-LSTM`	macro-F1	0.682	acc. 0.834	881,219	278.4	1127
`EC-PT-LSTM`	macro-F1	0.695	acc. 0.829	882,499	188.2	1126
`ST-TF-RNN`	RMSE (%)	0.6849	R^2 0.320	9,569	20.2	87
`ST-PT-RNN`	RMSE (%)	0.6956	R^2 0.299	9,729	31.1	127
`ST-TF-LSTM`	RMSE (%)	0.6788	R^2 0.332	38,177	86.5	362
`ST-PT-LSTM`	RMSE (%)	0.6828	R^2 0.324	38,817	66.3	264

Key findings

1. **EC: the LSTM wins on the hard class, not on accuracy.** Accuracy is nearly identical for all four runs (0.829–0.834) because 64 % of the reviews are positive. The 5-layer RNN almost never predicts *neutral* (F1 0.10–0.13); the LSTM does (F1 0.31–0.35), which raises macro-F1 from 0.605–0.614 to 0.682–0.695 and lowers the log-loss by about 0.04.

2. **EC: TF-IDF with logistic regression is still the benchmark.** It has the best accuracy (0.850) and log-loss (0.396), and a macro-F1 (0.688) equal to the LSTMs'. A bag of words that ignores order is hard to beat on this task.

3. **EC: long reviews do not favour the LSTM.** Accuracy by review length shows the LSTM ahead only for reviews of up to 20 words and slightly behind above 40 words. The expectation that memory pays off on long reviews is not supported here.

4. **ST: the networks beat a strong hand-built forecaster, by little.** All four runs reach R^2 = 0.30–0.33 out of sample against 0.30 for the EWMA (RiskMetrics) rule and 0.22 for the 20-day mean. The LSTM is ahead of the RNN by 0.9–1.8 % in RMSE, which is no larger than the 1.6 % spread between Keras and PyTorch for the *same* model. Only the two LSTMs beat the EWMA in every volatility regime and every test year.

5. **Cost.** The LSTM has four times the recurrent weights and needs 1.3–4.0× (EC) and 2.1–4.3× (ST) the time per epoch.

2 A Five-Layer RNN and a Five-Layer LSTM as Compositions of Functions

The assignment asks for the structure of a 5-layer RNN and a 5-layer LSTM written as compositions of elementary functions. This chapter builds both networks from five operations, checks every formula on numbers small enough to read (notebook `00_rnn_lstm_concepts.ipynb`), and explains why each piece of the design exists. Notation: a sequence has T steps; the input at step t is a row vector $x_t \in \mathbb{R}^F$; a layer has H hidden units; layers are numbered $l = 1, \ldots, L$ with $L = 5$; $h_t^{(l)}$ is the hidden state of layer l at step t. Vectors are rows, so a layer is written xW.

2.1 The Five Elementary Functions

Table 2: The operations from which both networks are built, with the numbers used in the notebook.

Function	Formula	Example
matrix product	$(xW)_j = \sum_i x_i W_{ij}$	$(2,1)\begin{pmatrix}1 & 2\\ 0 & 1\end{pmatrix} = (2,5)$
addition	$a + b$	$(2,5) + (0.5,-1) = (2.5,4)$
tanh	$\tanh z = \dfrac{e^z - e^{-z}}{e^z + e^{-z}} \in (-1,1)$	$\tanh(2.5) = 0.9866$, $\tanh(4) = 0.9993$
sigmoid	$\sigma(z) = \dfrac{1}{1+e^{-z}} \in (0,1)$	$\sigma(0) = 0.5$, $\sigma(2) = 0.8808$
element-wise product	$(a \odot b)_i = a_i b_i$	$(0.8, 0.2) \odot (5,5) = (4,1)$

Their derivatives decide how well a gradient travels: $\tanh'(z) = 1 - \tanh^2 z \le 1$ and $\sigma'(z) = \sigma(z)(1-\sigma(z)) \le 0.25$. A saturated unit passes almost nothing back: $\tanh'(3) = 1 - 0.9951^2 = 0.0099$.

Why these five.

- *Matrix product*: every hidden unit must see every input value, each with its own weight; one product computes all units at once.
- *Addition*: the cell merges two sources, the new input and its own memory. Addition keeps both and has derivative 1, so a gradient passes through it unchanged.
- *A nonlinearity*: without one, depth is useless, because $(xW_1)W_2 = x(W_1W_2)$: five linear layers equal one matrix.
- tanh *instead of ReLU in a recurrence*: the same W_h is applied at every step. If it scales the state by 1.1, an unbounded ReLU state grows $1.1^{40} \approx 45$-fold in 40 steps, while tanh pushes every step back into $(-1,1)$ and is zero-centred.
- *Sigmoid for gates*: a gate is a fraction between "block" (0) and "pass" (1), which is exactly the range of σ; tanh also takes negative values and cannot serve.
- $\odot$: a gate acts on every unit separately (keep unit 1, forget unit 2), so gate and content are multiplied position by position, not with a full matrix.

2.2 One RNN Cell

At step t the cell combines the new input with its previous state:

$$h_t = \tanh\big(x_t W_x + h_{t-1} W_h + b\big), \qquad h_0 = 0. \tag{1}$$

As a composition, $h_t = \tanh \circ \mathrm{add}\,(x_t W_x,\ h_{t-1} W_h,\ b)$: two matrix products, one addition, one tanh. The same W_x, W_h, b are used at every step (weight sharing), so the cell is one function applied T times, each time feeding its output back in. Weight sharing is not only an economy: the rule for

reading a word should not depend on where the word sits ("not good" means the same at word 3 and at word 80), and a single W_h serves a sequence of any length, whereas separate weights per step would multiply the parameters by T. Unrolled for three steps, $h_3 = \tanh\big(x_3W_x + \tanh(x_2W_x + \tanh(x_1W_x + b)W_h + b)W_h + b\big)$: the first input reaches h_3 only through nested W_h and tanh, which is the root of the gradient problem in Section 2.5.

Worked example ($F = H = 2$). Take $W_x = \begin{pmatrix} 0.6 & -0.4 \\ 0.3 & 0.9 \end{pmatrix}$, $W_h = \begin{pmatrix} 0.2 & 0.5 \\ -0.3 & 0.1 \end{pmatrix}$, $b = (0.1, -0.1)$ and the inputs $x_1 = (1, 0.5)$, $x_2 = (-0.5, 1)$, $x_3 = (0, -1)$. Step 1: $x_1W_x = (0.75,\ 0.05)$, $h_0W_h = 0$, so $a_1 = (0.85, -0.05)$ and $h_1 = \tanh(a_1) = (0.6911, -0.0500)$. Step 2: $x_2W_x = (0.00,\ 1.10)$ and $h_1W_h = (0.1532,\ 0.3406)$, so $a_2 = (0.00 + 0.1532 + 0.1,\ 1.10 + 0.3406 - 0.1) = (0.2532,\ 1.3406)$ and $h_2 = (0.2479, 0.8718)$. Step 3 gives $h_3 = (-0.3901, -0.6578)$. All numbers were produced by the notebook and agree with this hand calculation.

2.3 Stacking Five Layers

A 5-layer RNN stacks five such cells. Layer 1 reads the input sequence; layer l reads the *hidden states of layer* $l - 1$ as its input sequence:

$$h_t^{(1)} = \tanh\left(x_t W_x^{(1)} + h_{t-1}^{(1)} W_h^{(1)} + b^{(1)}\right), \tag{2}$$

$$h_t^{(l)} = \tanh\left(h_t^{(l-1)} W_x^{(l)} + h_{t-1}^{(l)} W_h^{(l)} + b^{(l)}\right), \qquad l = 2, \ldots, 5. \tag{3}$$

The prediction uses the top layer at the last step, $\hat{y} = g\big(h_T^{(5)} W_y + b_y\big)$, where g is a softmax for the three-class review task and the identity for the volatility regression. Calling $f^{(l)}$ the map that turns a whole sequence into the sequence of layer-l states, the network is the composition

$$\hat{y} = g \circ \text{last} \circ f^{(5)} \circ f^{(4)} \circ f^{(3)} \circ f^{(2)} \circ f^{(1)}\,(x_1, \ldots, x_T), \tag{4}$$

and every $f^{(l)}$ is itself T repetitions of matrix product $\to$ add $\to$ tanh. Figure 1 shows the grid: information moves *up* through the layers at one step and *right* through time inside one layer.

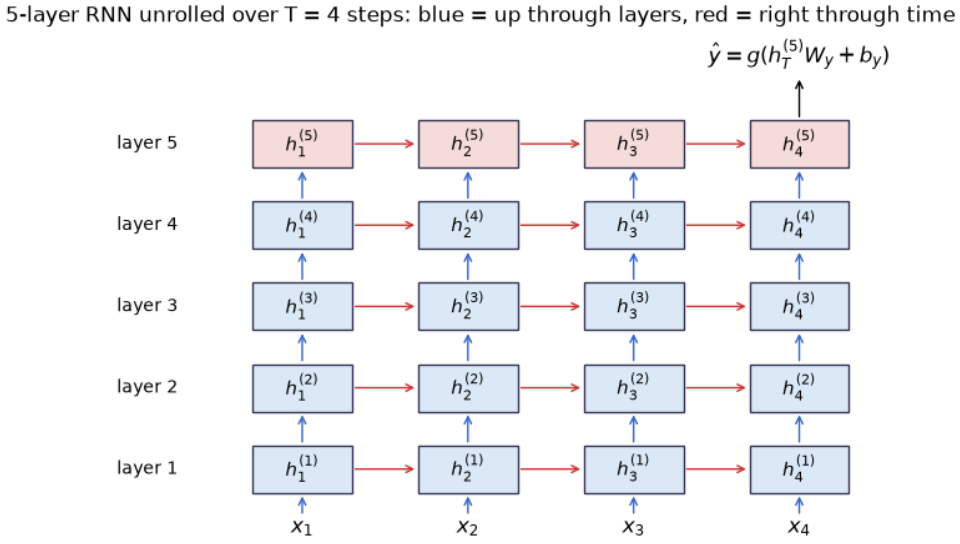


Figure 1: A 5-layer RNN unrolled over four steps. Blue arrows: up through layers; red arrows: right through time. The prediction reads only the top-right state.

Why stack, why the last step, why this output function. Each layer re-reads the sequence produced by the layer below, so higher layers work on more abstract summaries: in a review the first layer reacts to single words, a middle layer to short phrases ("not good"), the top layer to the overall verdict; in a price series the first layer sees one day, higher layers see weeks. Through the

recurrence $h_T^{(5)}$ depends on every input $x_1, \ldots, x_T$, so one vector at the last step summarises the sequence; each task needs one answer per sequence. A softmax output gives probabilities that sum to one and pairs with cross-entropy; the volatility is a real number trained with squared error, and squashing it would cap the forecast.

Shapes. The same network, shown as the shapes of the tensors it passes (batch of 256):

Step	EC (reviews)	ST (stocks)
input	$(256, 120)$ word ids	$(256, 40, 5)$ features
embedding	$(256, 120, 48)$	—
layers 1–5	each $(256, 120, 64)$	each $(256, 40, 32)$
last step of layer 5	$(256, 64)$	$(256, 32)$
output	$(256, 3)$ after softmax	$(256, 1)$

2.4 Counting Parameters

Layer 1 maps $F \to H$ and layers 2–5 map $H \to H$. With one bias per unit (the Keras convention) the 5-layer RNN with a head of K outputs has

$$\underbrace{FH + H^2 + H}_{\text{layer 1}} + \underbrace{4\,(H^2 + H^2 + H)}_{\text{layers 2–5}} + \underbrace{HK + K}_{\text{head}} \tag{5}$$

parameters. W_x has shape $F \times H$ (each unit has one weight per input feature) or $H \times H$ for the upper layers (their inputs are the H states below); W_h is $H \times H$ because every unit sees the previous state of all units in its layer. The count does not depend on T: a 10-word review and a 100-word review use the same weights. PyTorch stores two bias vectors (`bias_ih`, `bias_hh`) whose sum is the single bias of the formula, so it has $L \cdot H$ more parameters and exactly the same function. For the toy sizes $F = 3$, $H = 5$, $K = 1$ the formula gives $(15 + 25 + 5) + 4(25 + 25 + 5) + 6 = 271$; the Keras stack of five `SimpleRNN` layers plus head has 271 and PyTorch has $271 + 25 = 296$, as counted by the notebook.

Checking against the frameworks. The notebook implements the five-layer RNN in NumPy using only matrix product, addition and tanh, copies the weights of a PyTorch `nn.RNN(3,5,num_layers=5)` into it and into five Keras `SimpleRNN` layers, and compares the top-layer states of three random sequences of seven steps: the largest absolute difference is $8.3 \cdot 10^{-8}$ against PyTorch and $1.1 \cdot 10^{-7}$ against Keras, which is float32 round-off. Naming differs between frameworks: PyTorch keeps $W_x^\top$ as `weight_ih_l{k}`; Keras keeps W_x as `kernel` and W_h as `recurrent_kernel`.

2.5 The Gradient Through Depth and Time

Training needs $\partial \mathcal{L} / \partial h_t^{(l)}$ for every layer and step. By the chain rule, going back one step in time multiplies the gradient by

$$\frac{\partial h_t}{\partial h_{t-1}} = \operatorname{diag}(1 - h_t^2)\, W_h^\top, \qquad \frac{\partial h_T}{\partial h_t} = \prod_{k=t+1}^{T} \operatorname{diag}(1 - h_k^2)\, W_h^\top, \tag{6}$$

and going down one layer multiplies it by the same kind of factor with W_x. Reaching step t from step T takes $T - t$ factors, so the size of the gradient is roughly (typical factor)$^{T-t}$. If the typical factor is 0.6, then after 40 steps $0.6^{40} \approx 1.3 \cdot 10^{-9}$ and the gradient *vanishes*; if it is 1.2, then $1.2^{40} \approx 1{,}470$ and it *explodes*. Vanishing means the first words of a review hardly change the weights, so the RNN effectively learns from the last few steps only.

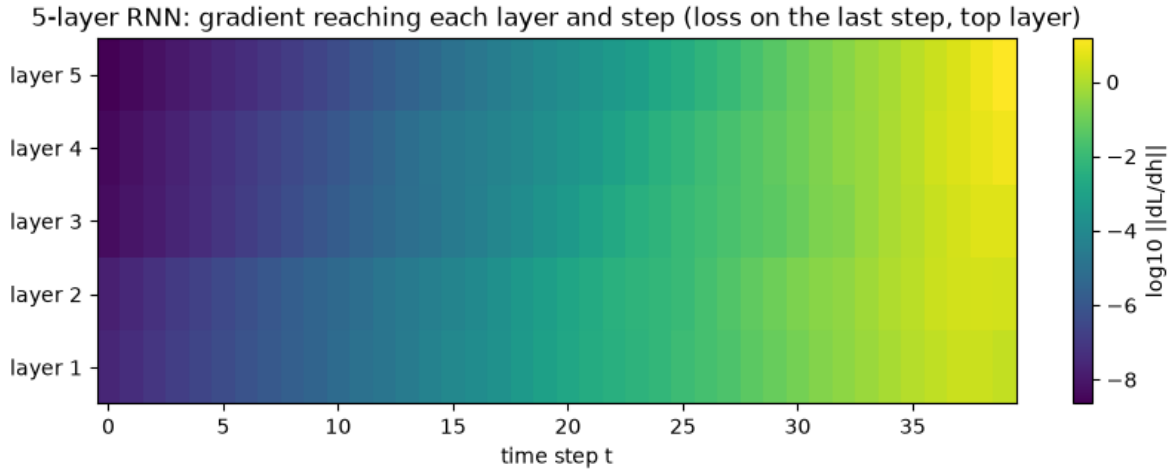


Figure 2: Size of $\|\partial \mathcal{L}/\partial h_t^{(l)}\|$ ($\log_{10}$) in a freshly initialised 5-layer RNN, $T = 40$, $H = 48$, loss on the last state of the top layer. Step 1 of layer 1 receives $1.7 \cdot 10^{-9}$ of the gradient at the top-right corner.

Exploding gradients and clipping. The opposite failure appears when the recurrent matrix amplifies. We set $W_h = \rho Q$ with Q orthogonal, unroll the 5-layer RNN for 40 steps and measure the gradient norm over all weights (median of five networks):

ρ	0.3	0.6	0.9	1.0	1.2	1.5	2	3	5
$\|g\|$	11.2	14.7	22.1	25.4	40.1	208	$1.3{\cdot}10^4$	$7.6{\cdot}10^6$	$2.2{\cdot}10^{10}$

Clipping by norm replaces g by $g \cdot \min(1, c/\|g\|)$: the direction is kept and the length is capped at c. With $c = 1$ a step $-\eta g$ can never exceed η in length, even at $\rho = 5$, where the raw gradient is $2.2 \cdot 10^{10}$. Clipping cures explosion only. A vanished gradient is already shorter than c, so clipping cannot enlarge it; that needs a different cell. All experiments in this report use $c = 1$.

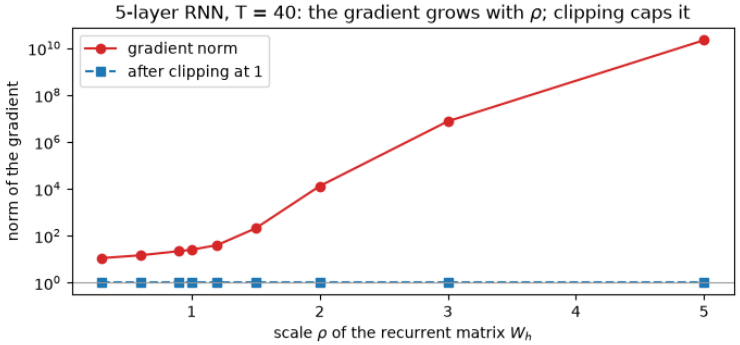


Figure 3: Gradient norm of the 5-layer RNN against the scale ρ of its recurrent matrix, before and after clipping at 1.

2.6 The LSTM Cell

An LSTM adds a second state, the *cell* c_t, and three *gates* that decide what to forget, what to write and what to show. Each gate is the RNN formula with a sigmoid in place of tanh:

Table 3: One LSTM step. Matrices $W_\bullet, U_\bullet$ act on x_t and h_{t-1}.

	Formula	Role
input gate	$i_t = \sigma(x_t W_i + h_{t-1} U_i + b_i)$	how much new content to write
forget gate	$f_t = \sigma(x_t W_f + h_{t-1} U_f + b_f)$	how much of the old cell to keep
candidate	$g_t = \tanh(x_t W_g + h_{t-1} U_g + b_g)$	the new content
output gate	$o_t = \sigma(x_t W_o + h_{t-1} U_o + b_o)$	how much of the cell to show
cell	$c_t = f_t \odot c_{t-1} + i_t \odot g_t$	memory, updated by addition
hidden	$h_t = o_t \odot \tanh(c_t)$	output of the step

The step $(h_t, c_t) = \mathrm{LSTM}(x_t, h_{t-1}, c_{t-1})$ uses matrix product, addition, sigmoid, tanh and $\odot$ and nothing else. A 5-layer LSTM stacks five of them exactly like the RNN: layer l reads $h_t^{(l-1)}$ and keeps its own cell $c_t^{(l)}$. **Worked example:** $f = 0.8$, $c_{t-1} = 1.5$, $i = 0.4$, $g = 0.5$ give $c_t = 0.8 \cdot 1.5 + 0.4 \cdot 0.5 = 1.2 + 0.2 = 1.4$.

Why each piece exists.

- The gates see x_t and h_{t-1}, because the decision to forget or to write depends on context: after reading "not", the word "good" must be treated differently, and only h_{t-1} knows that "not" came before.
- The candidate uses tanh because new content must be able to push a memory up or down.
- The cell is updated by *addition*, which keeps a copy of the old memory instead of squashing it again through a matrix and tanh. Hence $\dfrac{\partial c_t}{\partial c_{t-1}} \approx \mathrm{diag}(f_t)$: with $f = 0.97$ the signal over 40 steps is $0.97^{40} \approx 0.30$, whereas the RNN's typical $0.6^{40} \approx 1.3 \cdot 10^{-9}$ is eight orders of magnitude smaller.
- $h_t = o_t \odot \tanh(c_t)$ brings the unbounded sum back into $(-1, 1)$ for the next layer; the output gate lets the cell hold information without showing it until it is useful.
- Nobody sets the gates by hand; training adjusts W_f, U_f, b_f so that $f_t \approx 1$ where memory pays off and $f_t \approx 0$ where the state should reset.

Storage and parameters. Both frameworks store the four gates side by side in one matrix of width $4H$ in the order i, f, g, o (Keras calls g "c"): all four use the same inputs, so one large product is faster than four small ones. Each layer therefore holds four copies of the RNN's weights,

$$4(FH + H^2 + H) + 4 \cdot 4\,(2H^2 + H) + HK + K, \tag{7}$$

exactly four times the recurrent part of the RNN; the cell update and $h_t = o \odot \tanh c_t$ add no parameters. For $F = 3$, $H = 5$, $K = 1$ this is $180 + 880 + 6 = 1{,}066$ (Keras) and 1,166 (PyTorch, $+4LH = 100$ for the second bias). The same hand-versus-framework check as for the RNN gives maximum differences of $3.7 \cdot 10^{-8}$ (PyTorch) and $3.8 \cdot 10^{-8}$ (Keras).

2.7 Gradient Flow in the LSTM

Repeating the experiment of Figure 2 for the LSTM (Figure 4) depends on the initial forget-gate bias. At the start $f_t \approx \sigma(b_f)$ and the signal back through the cell is about f^T: with $b_f = 0$, $\sigma(0) = 0.5$ and $0.5^{40} \approx 9 \cdot 10^{-13}$, the cell begins by forgetting almost everything; with $b_f = 1$, $\sigma(1) = 0.73$ and $0.73^{40} \approx 3.4 \cdot 10^{-6}$, about $4 \cdot 10^6$ times more. Measured on the 5-layer network, step 1 of layer 1 receives $1.7 \cdot 10^{-9}$ (RNN), $4.1 \cdot 10^{-9}$ (LSTM, $b_f = 0$) and $9.5 \cdot 10^{-4}$ (LSTM, $b_f = 1$) of the gradient at the top-right corner. The principle is to start the network remembering and let it learn what to forget. Keras uses $b_f = 1$ by default, PyTorch uses 0, so our PyTorch models set it to 1 as well (Section 4).

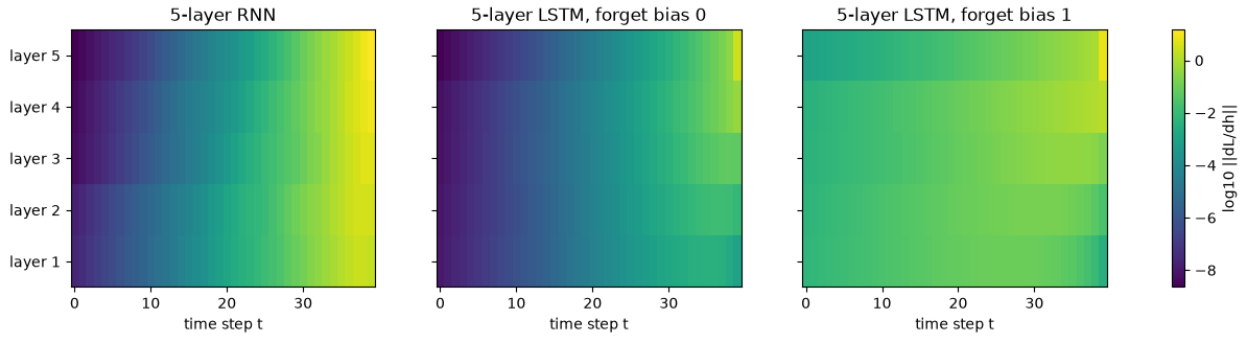


Figure 4: Same measurement as Figure 2 for the RNN and for the LSTM with forget bias 0 and 1 (common colour scale).

Table 4: The two architectures side by side.

	5-layer RNN	5-layer LSTM
elementary functions	matrix product, add, tanh	+ sigmoid, $\odot$
state per layer	h_t	h_t and cell c_t
parameters per layer	$FH + H^2 + H$	$4(FH + H^2 + H)$
gradient back one step	$\times \operatorname{diag}(1 - h^2) W_h^\top$: shrinks	through c: $\times f_t$, kept when $f_t \approx 1$
PyTorch / Keras	`nn.RNN(num_layers=5)` / 5 × `SimpleRNN`	`nn.LSTM(num_layers=5)` / 5 × `LSTM`

The LSTM's advantage is the cell path, which pays off only when the answer depends on information far back in the sequence. When the last few steps carry most of the signal, a plain RNN can do nearly as well at a quarter of the recurrent cost. The two experiments below test exactly this.

3 Data

Both datasets come from Kaggle, are about 100 MB, and were not used in any earlier assignment. They are chosen so that the two tasks stress the models differently: a sequence of words, where order and negation matter, and a sequence of days, where the signal is a slowly varying level of turbulence.

3.1 EC: Amazon Reviews of Unlocked Mobile Phones

Source. PromptCloud's *Amazon Reviews: Unlocked Mobile Phones*: 413,840 rows with product name, brand, price, a 1–5 star rating, the review text and the number of votes.

Cleaning. 70 reviews have no text and 251,280 further rows repeat a text already listed: the same review appears once per variant of a phone. Keeping copies would put the same text in the training and in the test set, so each text is kept once, which leaves 162,490 reviews.

Task. Three-way sentiment: *negative* (1–2 stars, 27.3 %), *neutral* (3 stars, 8.8 %) and *positive* (4–5 stars, 63.9 %). The 3-star class is kept deliberately. It is the hardest case for a network that reads one word at a time ("works fine, but…") and the case in which memory should matter most.

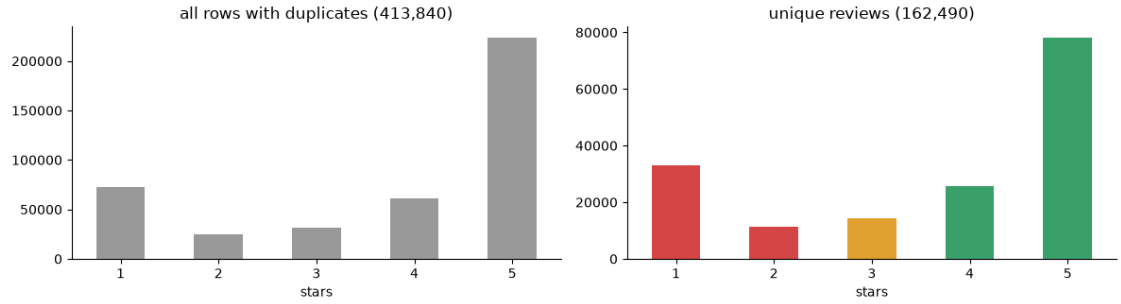


Figure 5: Star ratings before (left) and after (right) removing duplicate texts. Five stars dominates and one star comes second: people write when they are delighted or angry.

Length. The median review has 31 words if it is negative or neutral and 21 if it is positive (75th percentile: 62, 65 and 46). The less happy the customer, the more there is to explain, so evidence for the two minority classes is often spread over many words. 91.0 % of the reviews have at most 120 words.

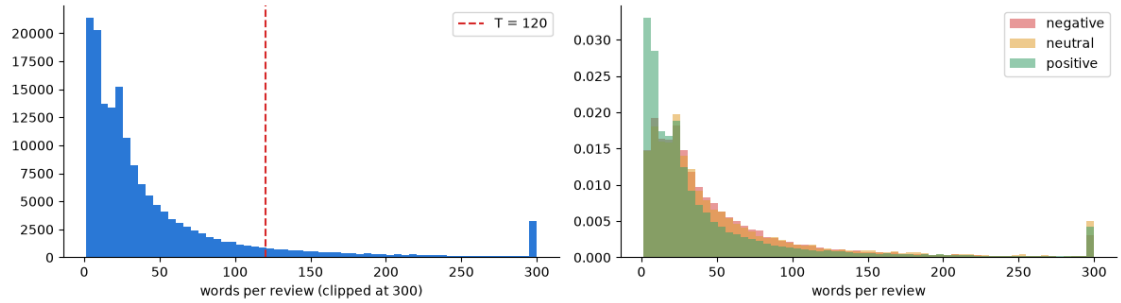


Figure 6: Review length. Left: all reviews, with the sequence length $T = 120$ used by the networks. Right: by class (normalised).

From text to numbers. A network cannot read text, so each review goes through four steps: (1) *tokenize* into lower-case words (letters, digits and apostrophes); (2) build a *vocabulary* of the 15,000 most frequent words of the training set, with id 0 for padding and id 1 for unknown words; (3) *encode* each word as its id; (4) *pad or truncate* to $T = 120$. Reviews shorter than 120 words receive zeros in front (pre-padding), so the last steps the network reads are real words and the final state h_T has just seen the text. A longer review keeps its *last* 120 words, because people state their verdict at the end ("so I returned it") and the final state is the one that is classified. The data are split *before* the vocabulary is built (stratified 80/10/10: 129,992 training, 16,249 validation and 16,249 test reviews), so validation and test words never enter the vocabulary. The training set has 62,418 distinct words; the top 15,000 cover 99.0 % of all word occurrences, and 1.02 % of the tokens become `<unk>`.

Class weights. With 64/27/9 % of the reviews, plain cross-entropy lets the network ignore the neutral class. Each training review is weighted by $w_c \propto 1/\sqrt{\text{share}_c}$, normalised to mean 1 over the training set, which gives 1.182 (negative), 2.077 (neutral) and 0.773 (positive). The square root is a compromise: full inverse-frequency weights would make one neutral review count seven times as much as a positive one and make training noisy. The weights change only the *training* loss; validation loss, early stopping and all metrics are unweighted, so every run is judged on the real class mix.

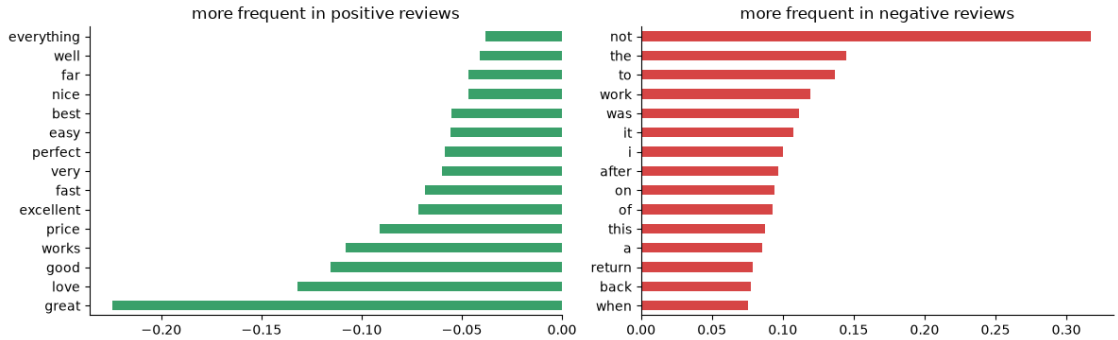


Figure 7: Words whose document frequency differs most between positive and negative reviews, before any model is trained (60,000 reviews, 3,000 most common words).

3.2 ST: S&P 500 Stocks

Source. *S&P 500 Stocks* (andrewmvd): daily open, high, low, close, adjusted close and volume of 502 stocks, 2010-01-04 to 2024-12-20, 1,891,536 rows. Rows before a company joined the index are empty, so only the **145 stocks with all 3,768 trading days** are used.

Task. For stock s and day t, the input is the last $T = 40$ days and the target is

$$y_t = \frac{1}{5}\sum_{k=1}^{5}|r_{t+k}|, \qquad r_t = \ln\frac{P_t}{P_{t-1}}, \tag{8}$$

the mean absolute move of the next five trading days (a one-week volatility forecast; P_t is the adjusted close). *Why volatility and not direction?* Daily returns are close to a random walk: the lag-1 autocorrelation of r_t is -0.040, whereas for $|r_t|$ it is 0.215. The *size* of moves clusters in time (calm days follow calm days), so there is structure to learn. *Why five days and not one?* One day's $|r|$ is a very noisy target. A weekly mean keeps the level of turbulence and removes much of the noise: measured on non-overlapping 5-day blocks, the autocorrelation one week apart is 0.477 for the weekly mean against 0.178 for the daily value at the same distance, and 0.196 against 0.077 eight weeks apart (Figure 8). (The 5-day blocks must not overlap: a rolling 5-day mean shares four of its five days with its neighbour and would show a spuriously high autocorrelation of 0.9.)

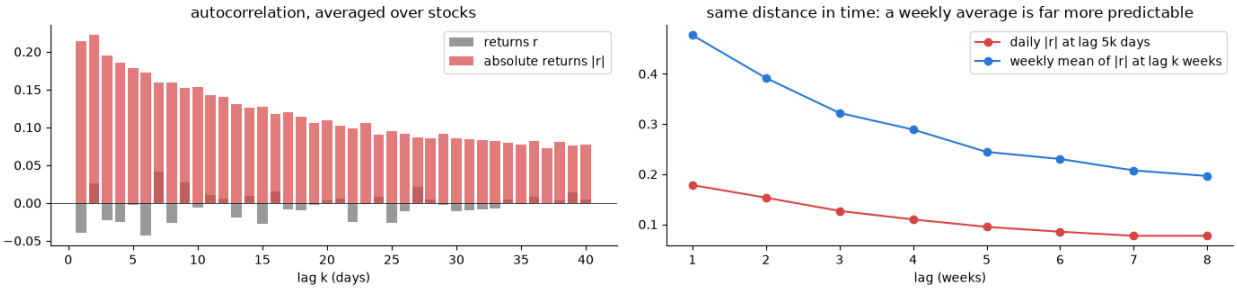


Figure 8: Left: autocorrelation of returns (grey, near 0) and absolute returns (red, positive for 40 days), averaged over the 145 stocks. Right: at equal distance in time, a weekly average of $|r|$ (non-overlapping blocks) is much more strongly tied to its past than a single day.

Features (five per day, all z-scored with training-period statistics): the log return r_t; the high–low range (High − Low)/Close; the open–close return ln(Close/Open); the log volume (z-scored per stock); and the *market return*, the mean of r_t over all 145 stocks on day t. The last one is the only feature that is not about the stock itself: when the whole market moves, single stocks follow, and a market-wide shock is the main way turbulence spreads. It uses day-t information only.

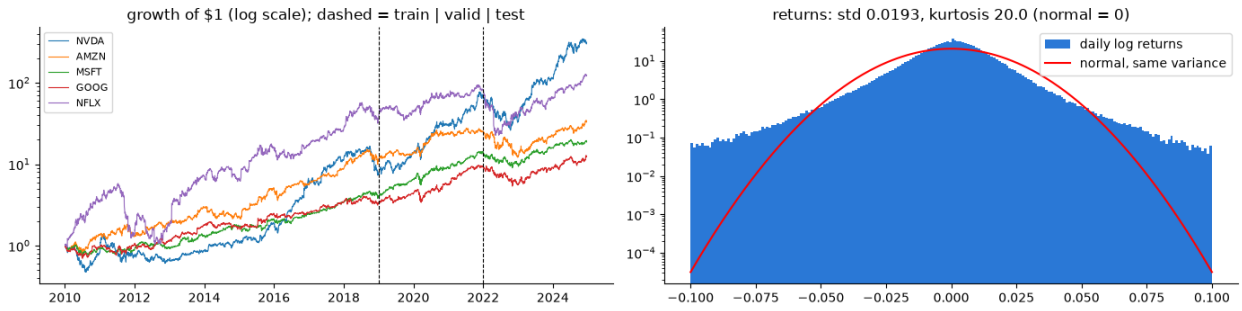


Figure 9: Left: growth of $1 in the five most traded stocks (log scale); dashed lines are the split dates. Right: daily returns have fat tails compared with a normal distribution of the same variance.

Split. Chronological and by calendar: training windows are those whose target ends before 2019 (321,755 windows), validation 2019–2021 (109,765) and test 2022–2024 (108,315). A window belongs to the split of the day its *target ends*, so no target crosses a boundary: with a 5-day target, a window ending on the last training day would have a target reaching into the validation period, and it is dropped from training. The *input* of a validation or test window may look back into the previous period, as it would in real use. Scaling uses training-period statistics only. The three test years are different markets: 2022 was a bear market with rising interest rates (mean target 1.67 %/day), 2023 a recovery (1.24 %) and 2024 a calm year (1.17 %).

4 Models, Protocol and Implementation

4.1 Architecture and Hyperparameters

Table 5: The protocol shared by the four runs of each dataset.

	EC (reviews)	ST (stocks)
input	120 word ids → embedding 48	40 days × 5 features
recurrent part	5 layers, $H = 64$	5 layers, $H = 32$
output	Dense(3) + softmax	Dense(1), linear
loss	class-weighted cross-entropy	mean squared error, standardised target
optimizer	Adam, gradient clipping at norm 1	
learning rate / batch	10^{-3} / 256	$2 \cdot 10^{-3}$ / 512
epochs / early stopping	≤ 8, patience 2	≤ 12, patience 3
both: validation loss decides, the best epoch is restored; same arrays for Keras and PyTorch		
seed	7	2024
parameters (RNN / LSTM, Keras)	760,451 / 881,219	9,569 / 38,177

For EC the embedding table (15,000 × 48 = 720,000 parameters) holds 95 % of the RNN's parameters and 82 % of the LSTM's; the five recurrent layers hold 40,256 (RNN) and 161,024 (LSTM) parameters. The counts follow the formulas of Section 2: for example the ST RNN has $(5 \cdot 32 + 32^2 + 32) + 4(2 \cdot 32^2 + 32) + 33 = 9{,}569$ parameters.

4.2 Keras and PyTorch

In Keras the 5-layer network is a list of layers; in PyTorch one call with `num_layers=5` is the whole stack $f^{(5)} \circ \cdots \circ f^{(1)}$, and the training loop is written by hand:

```
# Keras: four layers return the whole sequence, the fifth only its last state
stack = [layers.Input((MAXLEN,)), layers.Embedding(VOCAB, EMB)]
stack += [cell(H, return_sequences=(l < LAYERS - 1)) for l in range(LAYERS)]
stack += [layers.Dense(3, activation="softmax")] # cell = SimpleRNN or LSTM
```

```
# PyTorch: nn.RNN / nn.LSTM with num_layers=5, then the last step of the top layer
class TorchSentiment(nn.Module):
    def __init__(self, kind):
        self.emb = nn.Embedding(VOCAB, EMB)
        self.rnn = {"RNN": nn.RNN, "LSTM": nn.LSTM}[kind](EMB, H, num_layers=5,
            batch_first=True)
        self.head = nn.Linear(H, 3)
    def forward(self, x):
        out, _ = self.rnn(self.emb(x))
        return self.head(out[:, -1]) # logits from h^(5)_T
```

Making the two frameworks start alike. Keras and PyTorch initialise differently, which would make a framework comparison partly a comparison of initialisations. The PyTorch weights are therefore set to Keras's defaults: Glorot-uniform input weights (per gate for the LSTM), orthogonal recurrent weights, zero biases, forget-gate bias 1, embedding uniform in ± 0.05. PyTorch stores two biases per layer, so its models have $L \cdot H$ (RNN) or $4L \cdot H$ (LSTM) more parameters (EC: +320 and +1,280; ST: +160 and +640), all starting at zero; their sum is Keras's single bias, so the function is the same. The class weights enter PyTorch as the per-review loss times w_y, averaged over the batch, which is exactly what Keras's `class_weight` does.

Verification. Section 2 checks the equivalence on a small network: the NumPy implementation built from the five elementary functions agrees with PyTorch and Keras to $1.1 \cdot 10^{-7}$ (RNN) and $3.8 \cdot 10^{-8}$ (LSTM), and the parameter counts follow the formulas (271 and 1,066 for Keras; 296 and 1,166 for PyTorch). Each dataset notebook also asserts the expected parameter count before training.

4.3 Baselines and Metrics

EC. The *class prior* (always predict the training class shares, hence always "positive") is the floor. *TF-IDF + logistic regression* (unigrams and bigrams, 200,000 features, multinomial, the same class weights) is a strong classical model that ignores order: if a recurrent network cannot beat it, its ability to read order is not being used. Metrics: accuracy; the F1 of each class and macro-F1, their plain mean (so the 9 % neutral class counts as much as the 64 % positive one); one-vs-rest ROC-AUC averaged over classes; and the log-loss, which also judges how well calibrated the probabilities are.

ST. All baselines use only the past: the *train mean*; the mean of the *last 5 days* (persistence); the mean of the *last 20 days*; and the *EWMA* $\sigma_t = 0.94\,\sigma_{t-1} + 0.06|r_t|$, the RiskMetrics rule. The EWMA is a recurrent model with a single number of state and a hand-chosen constant, which makes it the natural opponent of a learned recurrent network. Metrics in the original unit (% per day): RMSE, MAE, the out-of-sample $R^2 = 1 - \sum(y-\hat{y})^2 / \sum(y-\bar{y}_{\text{train}})^2$ (0 means no better than the train mean) and the correlation of forecast and truth.

5 Experiment 1: Three-Way Sentiment of Reviews (EC)

Table 6: EC, test set (16,249 reviews). F1 is per class; epochs = epochs run (best epoch); s/epoch = median time per epoch.

Run	acc.	F1 macro	F1 neg.	F1 neu.	F1 pos.	AUC	log-loss	params	s/epoch	epochs (best)
class prior	0.639	0.260	0.000	0.000	0.780	0.500	0.855	–	–	–
tfidf_logreg	0.850	0.688	0.834	0.305	0.924	0.916	0.396	–	–	–
`EC-TF-RNN`	0.829	0.614	0.800	0.133	0.909	0.886	0.478	760,451	69.7	4 (2)
`EC-PT-RNN`	0.830	0.605	0.803	0.102	0.909	0.880	0.477	760,771	143.4	4 (2)
`EC-TF-LSTM`	0.834	0.682	0.826	0.309	0.912	0.910	0.436	881,219	278.4	4 (2)
`EC-PT-LSTM`	0.829	0.695	0.824	0.350	0.912	0.913	0.432	882,499	188.2	6 (4)

The LSTM finds the neutral reviews. The accuracies of the four runs differ by 0.005, but the confusion matrices (Figure 10, rows normalised) show why: the RNNs send 53 % of the neutral reviews to "negative" and 38–41 % to "positive" and recognise only 7–9 % of them. The LSTMs recognise 31 % (Keras) and 42 % (PyTorch) while losing 1 point (Keras) and 7 points (PyTorch) of recall on the negative class. Neutral reviews mix praise and complaint, so the verdict depends on how the two parts combine, which the final state must carry from earlier words. A plausible reason for the LSTM's advantage is that its gated cell can hold such information while the RNN's last state is dominated by the final words; we did not test this directly. The net effect is macro-F1 0.682–0.695 against 0.605–0.614.

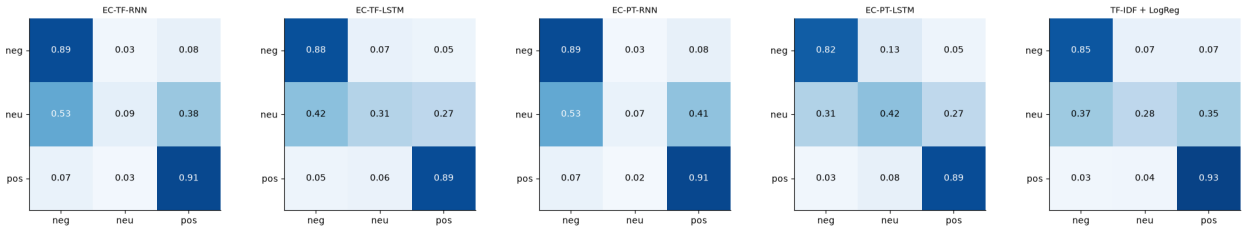


Figure 10: EC: confusion matrices on the test set, each row divided by its total (the diagonal is the per-class recall).

The star order is learned from three labels. The networks never saw the difference between 1 and 2 stars or between 4 and 5. Still, the score $P(\text{pos}) - P(\text{neg})$ rises monotonically with the rating for every model (Table 7), and is close to 0 at 3 stars for the PyTorch LSTM (−0.036) as for TF-IDF (−0.005). The LSTMs separate the extremes more strongly: at 5 stars they score 0.84–0.85 against 0.79 for the RNNs.

Table 7: EC: mean sentiment score $P(\text{pos}) - P(\text{neg})$ of the test reviews of each star rating.

stars	TF-RNN	TF-LSTM	PT-RNN	PT-LSTM	TF-IDF + LogReg
1	-0.657	-0.770	-0.668	-0.721	-0.739
2	-0.456	-0.527	-0.457	-0.449	-0.454
3	-0.081	-0.105	-0.056	-0.036	-0.005
4	0.540	0.557	0.553	0.596	0.630
5	0.785	0.842	0.792	0.853	0.867

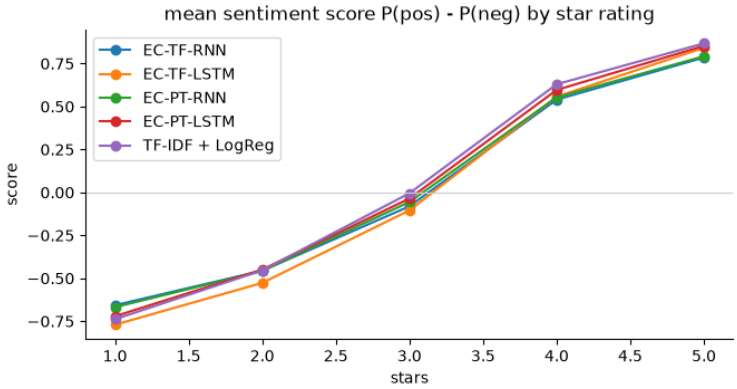


Figure 11: EC: the same scores as Table 7; the order of the stars is respected by every model.

Overfitting. Figure 12 shows validation loss reaching its minimum at epoch 2 (epoch 4 for the PyTorch LSTM) and rising afterwards while the training loss keeps falling: the 720,000-parameter embedding table is learned from 130,000 reviews and starts to memorise them. Early stopping

restores the best epoch. This is also why the LSTM's extra capacity does not translate into an accuracy gain.

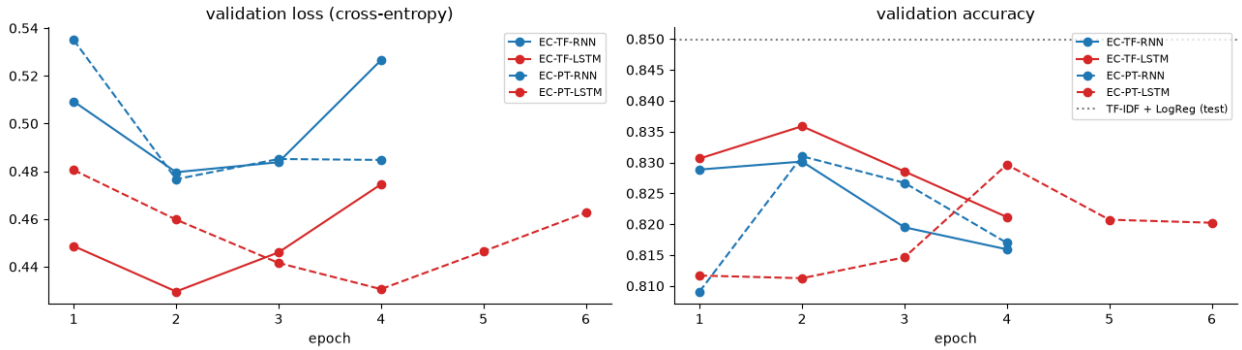


Figure 12: EC: validation loss (left) and validation accuracy (right) per epoch; dotted line: TF-IDF accuracy on the test set.

Long reviews do not favour the LSTM. If the RNN lost information over many steps, its accuracy should fall faster than the LSTM's on long reviews. Table 8 shows the opposite pattern: the LSTM is ahead by 0.9–1.4 points on reviews of up to 20 words, level at 21–40 words and up to 2 points behind above 40 words; above 120 words (cut to the last 120) every recurrent model loses clearly to TF-IDF (0.74–0.78 against 0.84), which sees every word. The vanishing-gradient argument of Section 2.5 concerns training signal for *early* words, and on these data it does not visibly limit the RNN. A possible reason, which we did not test, is that with pre-padding and a verdict at the end most of the information is in the last few steps. What the data show is that the LSTM's gain on EC comes from the neutral class, not from long reviews.

Table 8: EC: test accuracy by review length. Reviews above 120 words are cut to their last 120.

words	reviews	TF-RNN	TF-LSTM	PT-RNN	PT-LSTM	TF-IDF + LogReg
1-10	4,159	0.872	0.886	0.873	0.882	0.880
11-20	2,616	0.826	0.838	0.835	0.845	0.851
21-40	4,130	0.827	0.830	0.834	0.834	0.850
41-80	2,852	0.806	0.804	0.806	0.790	0.824
81-120	1,053	0.807	0.805	0.802	0.782	0.819
>120 (cut)	1,439	0.776	0.771	0.755	0.742	0.837

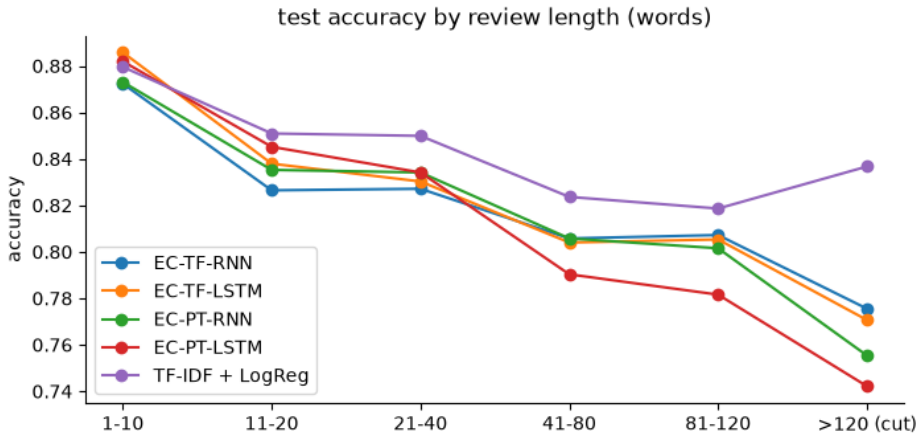


Figure 13: EC: test accuracy by review length, all runs and TF-IDF.

TF-IDF remains the benchmark. It has the best accuracy (0.850) and log-loss (0.396), is level

on AUC (0.916 against 0.910–0.913) and on macro-F1 (0.688 against 0.682 and 0.695; the two LSTM implementations differ by 0.013, more than their gap to TF-IDF). Sentiment in product reviews is largely lexical ("great", "broke", "refund"); a bag of words sees all words of every review, while the networks see at most 120 and must learn their embeddings from scratch on 130,000 reviews. We state this as measured: with this training set and no pre-trained embeddings, the recurrent models match a linear baseline on macro-F1 but do not beat it.

6 Experiment 2: One-Week Stock Volatility (ST)

Table 9: ST, test set (108,315 windows, 2022–2024). Errors are in % per day; baselines use only the past.

Run	RMSE (%)	MAE (%)	R^2	corr.	params	s/epoch	epochs (best)
train mean	0.8307	0.5613	0.000	–	–	–	–
last 5 days	0.8819	0.6175	-0.127	0.404	–	–	–
last 20 days	0.7332	0.5174	0.221	0.485	–	–	–
EWMA 0.94	0.6969	0.4894	0.296	0.526	–	–	–
`ST-TF-RNN`	0.6849	0.4821	0.320	0.533	9,569	20.2	4 (1)
`ST-PT-RNN`	0.6956	0.5005	0.299	0.516	9,729	31.1	4 (1)
`ST-TF-LSTM`	0.6788	0.4758	0.332	0.544	38,177	86.5	4 (1)
`ST-PT-LSTM`	0.6828	0.4838	0.324	0.540	38,817	66.3	4 (1)

The baselines show what a model has to beat. The mean of the last 5 days is *worse* than the train mean ($R^2 = -0.127$): five days are a noisy copy of next week's level. Averaging 20 days removes noise and gives $R^2 = 0.221$; the EWMA, which weights recent days more and old days less without a hard cut-off, gives 0.296.

The networks beat the EWMA, by a small margin. The four runs reach R^2 between 0.299 and 0.332. The RMSE gain over the EWMA is 0.2 % (PyTorch RNN) to 2.6 % (Keras LSTM), and the correlation with the truth is 0.516–0.544 against 0.526 for the EWMA. Figure 14 shows what this looks like: all forecasts follow the level of turbulence but none anticipates a spike, as expected of a volatility forecast. The networks have additional inputs (range, volume, market return) and learn the weights of the past days instead of fixing 0.94; yet a rule with one hand-chosen constant is already within 3 % of them in RMSE.

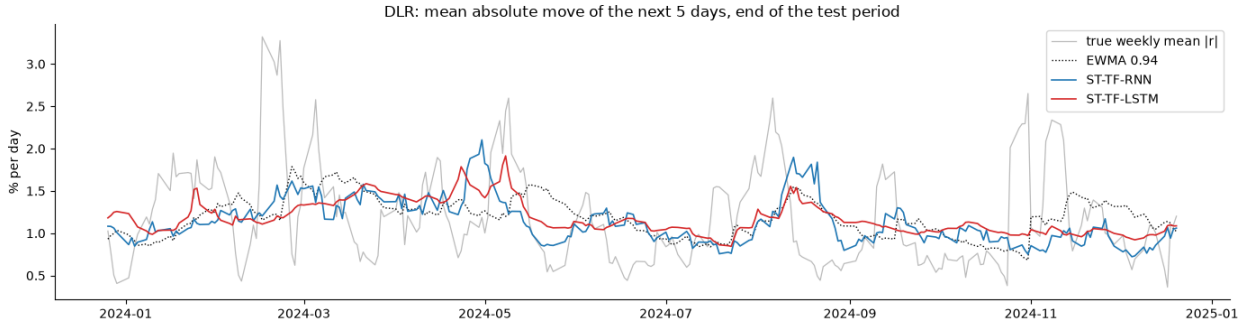


Figure 14: ST: true weekly mean $|r|$ and forecasts for one stock (DLR, from the middle of the liquidity ranking), last 250 test days.

Early stopping picked epoch 1 in all four runs. Validation loss is about 1.8 (standardised MSE) against 0.6 on the training set and rises from epoch 2 (Figure 15). The validation period 2019–2021 contains the crash of March 2020, whose very large targets dominate the loss, so the

validation loss mostly measures how a model copes with a regime the training period (2010–2018) never contained, not how well it has fitted the training data. The reported models are therefore barely trained. This is a limitation of the protocol, discussed in Section 8; the test results are still valid, because the test set was not used for any decision.

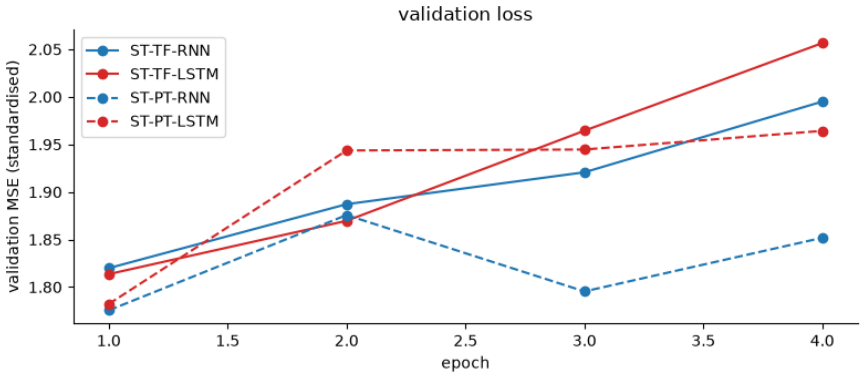


Figure 15: ST: validation loss per epoch.

Where does the LSTM help? Table 10 splits the test windows into five groups by the 20-day volatility at prediction time, and Table 11 by test year. Both LSTMs beat the EWMA in all five regimes and all three years. The RNNs do not: they lose narrowly in the calm and middle regimes, and the PyTorch RNN ties the EWMA in 2022 and is slightly worse in 2023. All models are least accurate in the most turbulent group (RMSE 0.91–0.94 % against 0.49–0.50 % in the calmest), since the level of the target rises and falls together with its error.

Table 10: ST: test RMSE (%) by volatility regime (quintiles of the 20-day mean $|r|$ at prediction time).

20-day volatility	TF-RNN	TF-LSTM	PT-RNN	PT-LSTM	EWMA 0.94
calmest	0.500	0.491	0.504	0.497	0.511
calm	0.589	0.572	0.592	0.581	0.586
middle	0.650	0.634	0.657	0.641	0.648
turbulent	0.700	0.687	0.706	0.701	0.709
most turbulent	0.914	0.928	0.940	0.919	0.949

Table 11: ST: test RMSE (%) by year; the last column is the mean target in that year.

year	TF-RNN	TF-LSTM	PT-RNN	PT-LSTM	EWMA 0.94	mean target (%)
2022	0.764	0.762	0.782	0.768	0.782	1.667
2023	0.637	0.633	0.651	0.642	0.650	1.243
2024	0.645	0.632	0.644	0.628	0.650	1.173

LSTM against RNN: a small edge, within the noise. The LSTM's RMSE is lower by 0.9 % (Keras) and 1.8 % (PyTorch), but the two implementations of the *same* model differ by 1.6 % (RNN) and 0.6 % (LSTM). With one run per configuration the LSTM advantage is not larger than the spread between single runs, so we cannot call it established. It is also consistent with the data: the autocorrelation of the weekly level decays over weeks (Figure 8), so the information is in the recent past, within reach of a plain RNN; the cell path matters when the answer sits far back.

7 Comparison: LSTM against RNN, Keras against PyTorch

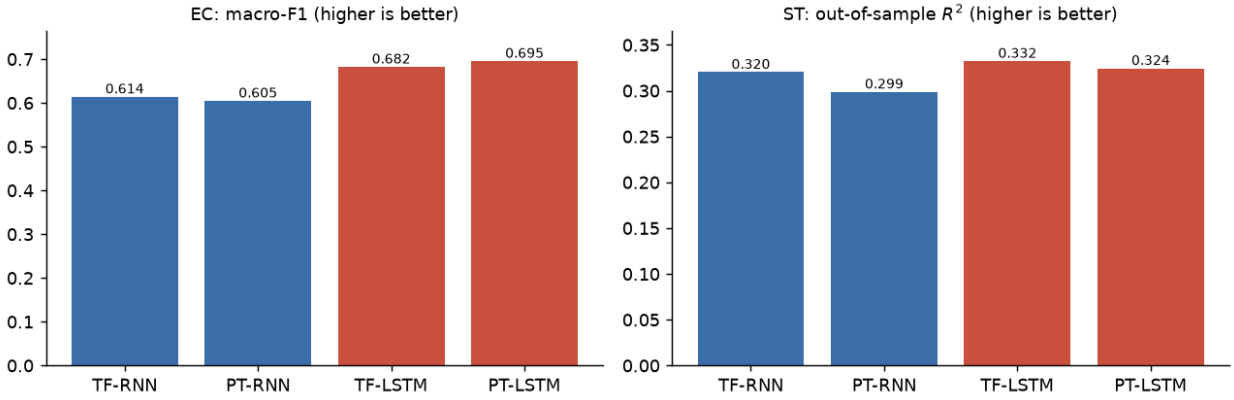


Figure 16: Main score of the eight runs (axes start at 0).

Accuracy. The LSTM's advantage differs by task and by metric. On EC it is large for macro-F1 (+0.07 to +0.09) and for the neutral class, and absent for accuracy. On ST it is a 1–2 % lower RMSE. In neither case is it explained by long-range memory in the sense of Section 2.5; in both, the data put the information in the last steps.

Parameters. The recurrent part is exactly 4× larger (EC: 40,256 → 161,024; ST: 9,569 → 38,177 in total). On EC the embedding dominates, so the total grows by only 16 %; on ST the whole model grows 4×.

Time. Seconds per epoch on a CPU (Figure 17): the LSTM needs 1.3× (PyTorch) to 4.0× (Keras) the time of the RNN on EC and 2.1× (PyTorch) to 4.3× (Keras) on ST, because each step does four matrix products instead of one. Epoch times of the PyTorch RNN on EC varied between 97 and 206 s inside one run, so timing ratios on this machine are indicative only.

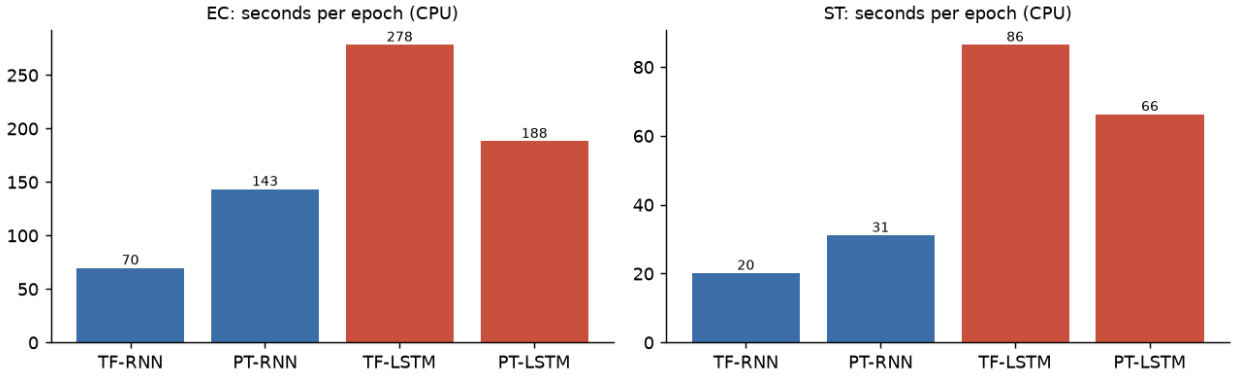


Figure 17: Median seconds per epoch (CPU only).

Framework. Keras and PyTorch, started from the same initialisation, land close together but not identically: macro-F1 0.614 / 0.605 (RNN) and 0.682 / 0.695 (LSTM); ST RMSE 0.685 / 0.696 % (RNN) and 0.679 / 0.683 % (LSTM). The remaining differences come from batch order, non-deterministic kernels and the epoch at which early stopping fires, not from different mathematics (the forward passes agree to 10^{-7}). This spread, about 0.01 in macro-F1 and 1 % in RMSE, is the yardstick for reading any difference between two single runs in this report.

8 Conclusions and Limitations

Conclusions.

- A 5-layer RNN is $\hat{y} = g \circ \text{last} \circ f^{(5)} \circ \cdots \circ f^{(1)}$ with each $f^{(l)}$ made of matrix products, additions and tanh; a 5-layer LSTM is the same grid with gates built from sigmoid, tanh and $\odot$. Hand-written versions agree with PyTorch and Keras to 10^{-7} and the parameter formulas match the frameworks exactly.
- Gradient clipping and the additive cell address the two opposite failures of recurrence: clipping caps an exploding gradient (up to $2 \cdot 10^{10}$ in our test), the cell path keeps a vanishing one alive (about $5 \cdot 10^{5}$ times more signal at the first step than in the RNN, with forget bias 1).
- On three-way review sentiment the LSTM is clearly better than the RNN on the neutral class and in macro-F1, equal in accuracy, and level with TF-IDF + logistic regression in macro-F1 but not better on accuracy or log-loss.
- On one-week stock volatility, networks of both types beat a strong EWMA baseline by 0.2–2.6 % in RMSE, and the LSTM's edge over the RNN is of the order of the run-to-run spread.
- The LSTM costs four times the recurrent weights and 1.3–4.3× the time per epoch. It is worth it when a minority class needs context (EC), and marginal when the signal is a short-memory level (ST).

Limitations.

- One run per configuration: no seeds, so no confidence intervals. Differences below about 0.01 in macro-F1 or 1 % in RMSE (the Keras–PyTorch spread) should not be read as real.
- ST validation period: 2019–2021 contains the March 2020 crash, so early stopping selected epoch 1 for all four runs. A validation period without such a shock, or time-series cross-validation, would train the models longer and may widen or close the gap between RNN and LSTM.
- Embeddings are learned from scratch on 130,000 reviews; pre-trained word vectors would likely lift the recurrent models over TF-IDF. Reviews are cut to the last 120 words.
- Fixed sizes ($H = 64$ and $H = 32$, five layers): both models were not tuned separately, and the RNN was not given four times the units to match the LSTM's parameters, so architecture and size are compared together.
- CPU-only timing on one machine; ratios between frameworks depend on thread settings and kernels.
- Not tried: bidirectional layers, GRU, attention, other horizons and targets for ST.

A Reproducing the Results

```
assignment_07/
  dataset/ec/Amazon_Unlocked_Mobile.csv (Kaggle: PromptCloudHQ/amazon-reviews-unlocked-
      mobile-phones)
  dataset/st/sp500_stocks.csv (Kaggle: andrewmvd/sp-500-stocks)
  notebook/00_rnn_lstm_concepts.ipynb Section 2: structure, parameter counts, gradients,
      checks
  notebook/ec_reviews.ipynb Experiment 1 (EC-TF-RNN, EC-TF-LSTM, EC-PT-RNN, EC-PT-LSTM)
  notebook/st_stocks.ipynb Experiment 2 (ST-TF-RNN, ST-TF-LSTM, ST-PT-RNN, ST-PT-LSTM)
  notebook/results/<run>.json, *_comparison.csv, EC_*.csv
  report/make_tables.py builds table_*.tex and cmp_*.png from the results
  report/Assignment_07.tex this report (XeLaTeX)
```

Setup and execution (Python 3.12; the report was produced with Python 3.13, TensorFlow 2.21, Keras 3.15 and PyTorch 2.14 on a CPU under Windows):

```
pip install -r requirements.txt
cd notebook
jupyter nbconvert --to notebook --execute --inplace 00_rnn_lstm_concepts.ipynb
jupyter nbconvert --to notebook --execute --inplace st_stocks.ipynb # about 15 min on a
    CPU
jupyter nbconvert --to notebook --execute --inplace ec_reviews.ipynb # about 55 min on a
    CPU
cd ../report && python make_tables.py && xelatex Assignment_07 && xelatex Assignment_07
```